

OpenHDID – Ügyfélszolgálati hívóazonosítás

A konténeres OpenHDID napi üzemeltetése: ellenőrzés, szolgáltatások kezelése, naplók és felügyelet, ütemezett feladatok, a leggyakoribb helyzetek, mentés és visszaállítás, frissítés, skálázás és biztonsági karbantartás.

VERZIÓ KIADÁS
1.4.2 · 2026. október 9. OpenHDID 1.4.2 · ghcr.io/adamfhun/openhdid

CÉLKÖZÖNSÉG KAPCSOLÓDÓ
rendszerüzemeltető, rendszergazda telepítési útmutató, felhasználói kézikönyv

TARTALOM

- 1 Napi és heti ellenőrzés
- 2 Szolgáltatások kezelése
- 3 Naplók és felügyelet
- 4 Ütemezett feladatok
- 5 Beállítások és titkok módosítása
- 6 Üzemeltetési helyzetek
- 7 Mentés és visszaállítás
- 8 Frissítés és visszaállítás
- 9 Skálázás és teljesítmény
- 10 Biztonsági karbantartás
- 11 Adatmegőrzés
- 12 Parancsreferencia
- R Rövidítésjegyzék

Minden parancs a telepítési mappából (például `/opt/openhdid`) futtatandó, ahol a `compose.yaml`, a `.env` és a `db.env` található. Az alkalmazás parancsainak előtagja `docker compose exec web php artisan`.

1 Napi és heti ellenőrzés

A legtöbb kérdésre az *Adminisztráció* › *Rendszer* › *Rendszerállapot* oldal válaszol. Minden összetevő állapota színes jelvénnel látszik, lenyitható tanáccsal; a fejléc a futó verziót mutatja. Ugyanezek az ellenőrzések gépi úton:

- `GET /health` : 200, ha semmi nem hibás, 503, ha bármi hibás; csak a felügyeleti címekről érhető el, lásd `OPENHDID_MONITORING_ALLOW` ;
- `docker compose exec web php artisan hdid:health` : kilépési kód 0 rendben, 1 figyelmeztetés (csak `--strict` mellett), 2 hiba; `--json` géppel olvasható.

Naponta:

- `docker compose ps` : minden szolgáltatás fut, a `web` `healthy` , a `migrate` `exited (0)` ;
- a Rendszerállapot kártyái zöldek:
 - sárga: naplózó levelező, élő csomag-felülírás, több ügyfélnél rögzített („megosztott”) telefonszám, hiányzó API-kulcs, a panelről indított, 5 percnél régebben várakozó átvétel, az „Ügyféltörzs-átvétel” kártyán a következő átvételnél lezáruló fiókok (a tételük az utolsó futásból hiányzott: az Ügyféltörzs (EMD) lista „A következő átvételnél lezárul” fülén ellenőrizhető, mielőtt a következő futás lezárja őket), vagy 120 és 300 mp közötti PHP időkorlát;
 - piros: nem futó feldolgozó vagy ütemező, adatbázis-hiba, élesben naplózó SMS, a várt időköz kétszeresénél régebbi utolsó sikeres átvétel, vagy 120 mp alatti PHP időkorlát (a képbe épített `php.ini` 300 mp-et ad; ennél rövidebb felülírás mellett az átvétel el sem indul, és a konténer indításkori ellenőrzése is megáll);
- *Kimenő üzenetek*: nincs „sikertelen” vagy órák óta „várakozik” tétel;
- *Ügyféltörzs-átvételek*: az utolsó átvétel sikeres, a lezárt tételek száma nem ugrott meg.

Hetente:

- az auditnapló `login.rejected` , `login.request_throttled` , `account.locked` , `api_key.rejected` (hálózati címenként és okonként percenként egy sor), `message.failed` és `message.refused` (kézbesítetlen levél vagy SMS, illetve az SMS-keret miatt elutasított belépőkód-kérés) és `id_session.manual` eseményei;
- a lemezterület: `docker system df` , és a kötetek mérete;
- a heti sérülékenységvizsgálat eredménye a kiadott image-en (10. fejezet).

2 Szolgáltatások kezelése

TEENDŐ	PARANCS
Állapot	<code>docker compose ps</code>
Indítás, módosítás érvényesítése	<code>docker compose up -d</code> (csak a változott konténereket indítja újra)
Egy szolgáltatás újraindítása	<code>docker compose restart web</code> (vagy <code>worker</code> , <code>sync-worker</code> , <code>scheduler</code>)
Leállítás (adatok megmaradnak)	<code>docker compose stop</code> , illetve <code>docker compose down</code> (a kötetek maradnak; a <code>-v</code> kapcsolót soha ne használja élesben)
Alkalmazásparancs	<code>docker compose exec web php artisan <parancs></code>
Parancs új, ideiglenes konténerben	<code>docker compose run --rm web artisan <parancs></code>

TEENDŐ	PARANCS
nginx újratöltése (tanúsítványcseré)	<code>docker compose exec web openhdid reload</code>
Tanúsítvány ellenőrzése	<code>docker compose exec web openhdid tls-check</code>
Verzió	<code>docker compose exec web openhdid version</code>
Karbantartási mód	<code>docker compose exec web php artisan down --retry=30</code> , majd <code>... up</code>

Újraindítás. A konténerek újraindítási szabálya `unless--stopped` : leállás vagy a gazdagép újraindítása után maguktól elindulnak. A `web` két folyamata (nginx és PHP-FPM) egymást felügyeli: ha az egyik leáll, a konténer is kilép, és a Docker újraindítja. Leállításkor mindkettő befejezi a futó kéréseket.

Karbantartási mód. `APP_MAINTENANCE_DRIVER=cache` mellett minden webpéldányra hat. Alatta a portál, a panel és a telefonközponti végpontok is „karbantartás” választ adnak; időzítse forgalmon kívülre.

3 Naplók és felügyelet

FORRÁS	HOL	MIRE JÓ
Webkiszolgáló	<code>docker compose logs web</code> : JSON access log (idő, IP, kérésazonosító, metódus, státusz, méret, időtartam), nginx- és PHP-FPM-hibák	forgalom, hibaarány, lassú kérések; URL, lekérdezés és süti nincs benne
Alkalmazás	<code>docker compose logs web worker sync-worker scheduler</code>	kivételek, figyelmeztetések (<code>LOG_LEVEL=warning</code>), naplózó módban a kimenő üzenetek
Indítás és beállítás	a konténerek első sorai: <code>openhdid:</code> előtaggal	beállítási hibák, tanúsítvány-összefoglaló, figyelmeztetések
Auditnapló	felület: <i>Rendszer</i> › <i>Auditnapló</i> (szűrők, CSV-export)	ki, mikor, mit: belépések, elutasítások, beállítások, azonosítások, kulcsok
Rendszerállapot	felület, <code>/health</code> , <code>hdid:health</code>	összetevők, életjelek, sorhossz, átvétel kora, megőrzés

```
docker compose logs -f --tail=200 web
docker compose logs --since 30m --no-log-prefix web | grep '"status":5'
docker compose logs --since 1d --no-log-prefix web | grep -v '^{'      # hibák,
figyelmeztetések
docker compose logs --no-log-prefix web | grep '"status":429' | wc -l  # forgalomkorlát-
elutasítások
```

Forgatás és gyűjtés.

- **Forgatás:** a naplókat a konténermotor forgatja (`json-file` meghajtó, 10 × 20 MB szolgáltatásonként; Podman alatt `k8s-file` , csak méretkorláttal, a `compose.yaml` `x-logging` blokkjában).
- **Központi gyűjtés:** ugyanitt állítható a `journald` , `syslog` , `gelf` vagy `fluentd` meghajtó. A napló titkosított csatornán, a futtatási fióktól elkülönített, módosítás ellen védett tárba kerüljön.
- **Hibasorok:** az nginx és az alkalmazás hibasoraiban URL vagy személyes adat lehet: korlátozott hozzáférés, rövid nyers megőrzés.

Külső felügyelet. Ajánlott figyelni:

- a `/health` választ (503 = riasztás);
- a konténerek egészségét (`docker compose ps` , `health_status` események);
- a lemezterületet és a tanúsítvány lejáratát (a web naplója 30 nappal előtte figyelmeztet);
- a kiugró 4xx/5xx arányt.

Egyszerű gazdagépes riasztás, ha nincs felügyeleti rendszer:

```
*/5 * * * * cd /opt/openhdid && docker compose exec -T web php artisan hdid:health
>/dev/null || echo "OpenHDID: hibás rendszerállapot" | mail -s "OpenHDID"
uzemeltetes@example.org
```

4 Ütemezett feladatok

A cron szerepét a `scheduler` szolgáltatás tölti be. A gazdagépen nincs crontab-bejegyzés, és a konténerekben sincs cron-démon. A feladatok listája és a következő futásuk:

```
docker compose exec scheduler php artisan schedule:list
```

- **Percenként:** lejárt azonosítási munkamenetek zárása; életjelek; elakadt üzenetek újrásorolása.
- **15 percenként:** ügyféltörzs-token frissítése, ha be van kapcsolva.
- **Óránként:** csomag-felülrások lejárata; befejezett hívások tisztítása; beragadt hívások lezárása.
- **Naponta 03:30-kor:** adatmegőrzési tisztítás.
- **Az `EMD_SYNC_CRON` szerint:** teljes ügyféltörzs-átvétel.

Időzóna: `APP_TIMEZONE` .

Az ütemező vagy a feldolgozó életjele hiányzik (piros „Ütemező” vagy „Feladatsor” kártya):

1. `docker compose ps scheduler worker` : fut-e; ha újraindulgat, `docker compose logs --tail=100 scheduler` . Jellemzően adatbázis-kapcsolat vagy beállítási hiba az ok, a konténer első soraiban.
2. A kártyák a következő percben zöldre válnak, amint az életjel megérkezik.
3. Az ütemezőből pontosan egy példány fusson; skálázáskor se növelje.

A panelről indított átvétel „várakozik” állapotban marad: a `sync-worker` nem fut, vagy az `EMD_SYNC_QUEUE` nem egyezik a figyelt sorral (alapból mindkettő `sync`). A Rendszerállapot 5 perc után figyelmeztet. A panelről feltöltött fájl is a `sync-worker` -ben dolgozódik fel (1.2.0 óta), ugyanezzel a

feltétellel.

Egy átvétel „Sikertelen”, a hibaszöveg szerint a PHP félbeszakította: a futást az időkorlát, a memória vagy egy összeomlás vágta el; a már beírt tételek megmaradtak, lezárás nem történt, a zár felszabadult. Nézze meg a `sync-worker` naplóját (`docker compose logs --tail=200 sync-worker`) és a Rendszerállapot „PHP időkorlát” kártyáját; a feldolgozó 1200 másodperces job-időkorlátja nagy állománynál is elegendő, ennél hosszabb futás a forrást vagy a memóriát jelzi.

5 Beállítások és titkok módosítása

- **Telepítési érték** (`.env`): szerkesztés, majd `docker compose up -d` . Az érintett konténerek újraindulnak, a konfigurációs gyorsítótár újraépül; ha egy érték hibás, a konténer nem indul el, az oka a naplóban.
- **Üzleti paraméter** (Beállítások oldal): azonnal él minden folyamatban, újraindítás nélkül.
- **Titokcsere** (adatbázis, EWS, Ozeki, SSO, ügyféltörzs): előbb a partner oldalán, majd a `.env` -ben (vagy a titokfájlban, `*_FILE`), végül `docker compose up -d` .
- **Az adatbázis-jelszó cseréje** a beépített MariaDB-ben:

```
docker compose exec mariadb sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb -uroot -e "ALTER USER \'openhdid\'@\'%\` IDENTIFIED BY \'új-jelszó\';"'
```

Utána a `.env` `DB_PASSWORD` értéke és `docker compose up -d` .

- **Az ügyféltörzs tokenjei** jelszó- vagy végpontcsere után: `docker compose exec web php artisan hdid:emd-sync-token` .
- **Hibakeresés részletes hibaüzenetekkel**. Éles környezetben (`APP_ENV=production`) az `APP_DEBUG=true` tiltott: a konténer 64-es kilépési kóddal megáll, mert a hibaoldal belső részleteket mutatna. Egy hibakeresési időszakra a `.env` -ben `APP_ENV=staging` és `APP_DEBUG=true` együtt, majd `docker compose up -d` ; a végén mindkettő visszaállítása és újra `docker compose up -d` . Ha csak több napló kell, az `APP_ENV` maradhat, elég a `LOG_LEVEL=debug` .

APP_KEY

Meglévő telepítésen nem cserélhető a szokásos módon. A titkosított válaszok, kódmásolatok, API-tokenek, sükik és kulcsolt lenyomatok mind tőle függnék; cseréje külön adatmigrációs tervet és visszaállítási próbát igényel.

6 Üzemeltetési helyzetek

Az alkalmazás működésével kapcsolatos helyzetek (belépés, azonosítás, hívások, jogosultságok) a felhasználói kézikönyvben és a Rendszerállapot kártyáinak tanácsaiban szerepelnek. Ez a fejezet a konténeres üzemre jellemző eseteket írja le.

Az összes diagnosztikai parancs egy helyen, másolható formában: a kiadás `TRUBLESHOOTING.md` mellékletében (állapot, újrainduló szolgáltatás, proxy, Docker-jogosultság, fájltulajdonos, ügyféltörzs-elérés és -belépés, tanúsítvány-lánc és belső CA, egyszeri bejelentkezés, a belépés elutasításának oka,

levél és SMS, mentés).

A konténer nem indul el vagy újraindulgat

1. `docker compose ps` és `docker compose logs --tail=50 <szolgáltatás>`. Az indítószkript `openhid: ERROR:` sora megnevezi az okot: hiányzó vagy hibás `APP_KEY`, nem https `APP_URL`, hiányzó adatbázis-adat, élesben tiltott beállítás, tanúsítványhiba, nem írható `storage` kötet.
2. Ha csak a `web` indul újra („restarting”), az ok a tanúsítvány vagy az nginx: a `docker compose run --rm --no-deps web tls-check` futó `web` nélkül is ellenőrzi a fájlokat (hiányzó `fullchain.pem` / `privkey.pem`, nem olvasható kulcs, névegyezés, csonka lánc). Az `artisan` parancsok ekkor is futnak: `docker compose run --rm --no-deps web artisan hdid:make-admin ...`.
3. A `migrate` hibájánál a többi szolgáltatás nem indul: `docker compose logs migrate`. Jellemző okok: az adatbázis nem érhető el 120 másodpercen belül, hibás migrációs jelszó a `db.env`-ben, vagy a migrációs fióknak nincs joga.
4. Javítás után: `docker compose up -d`.

Az image letöltése időtúllépéssel elhal

Ha a `docker compose pull` 30 másodperc után időtúllépést ad, miközben a `curl` a gazdagépről eléri a `ghcr.io`-t, a Docker-démon nem látja a felhasználó proxybeállítását (`env | grep -i proxy`). A démon a proxyt systemd drop-inből kapja:

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf >/dev/null <<EOF
[Service]
Environment="HTTP_PROXY=${http_proxy:-$HTTP_PROXY}"
"HTTPS_PROXY=${https_proxy:-$HTTPS_PROXY}"
"NO_PROXY=localhost,127.0.0.1,$no_proxy:-$NO_PROXY"
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
docker info | grep -i proxy
```

Ha nincs proxy, az IPv6 lehet félig beállítva: a `curl -4` és a `curl -6` a `https://ghcr.io/v2/` címre mutatja meg (HTTP/2 401 a jó válasz); az IPv4 előnyben részesítése a `/etc/gai.conf` `precedence ::ffff:0:0/96 100` sorával.

„permission denied ... docker.sock”

A futtató felhasználó nincs a `docker` csoportban, vagy a tagság még nem él: `sudo usermod -aG docker <felhasználó>`, majd új bejelentkezés (vagy `newgrp docker`); ellenőrzés `id` és `docker ps`. Ha a Docker kliens a Podman socketjét keresi, a korábbi `DOCKER_HOST` változót kell törölni (`unset DOCKER_HOST`, a profilból is), és a `docker context use default` parancsot futtatni.

Belépés elutasítva „hibás felhasználónév vagy jelszó” üzenettel

A felület minden elutasításra ugyanezt a semleges üzenetet adja. A valódi ok az auditnaplóban van (`login.rejected` esemény); a panel nélkül a konténerből kérdezhető le:

```
docker compose exec web php artisan tinker --execute 'App\Models\AuditLog::query()->where("event", "login.rejected")->latest()->take(5)->get()->each(fn ($e) => print($e->created_at." ".json_encode($e->context).PHP_EOL));'
```

A `reason` mező értékei: `invalid_credentials` (rossz e-mail vagy jelszó; a `hdid:make-admin` újrafuttatása ugyanarra az e-mailre a jelszót felülírja), `account_locked` (fokozódó zárolás, várni kell), `no_permission` (nincs panel-jogosultság), `method_disabled` (a jelszavas belépés kikapcsolva a beállításokban), `no_external_record` és `external_record_mismatch` (hiányzó vagy eltérő ügyféltörzs-tétel), `account_closed`. Ha egy sor sem keletkezett, a kérés el sem jutott a belépési szabályig: IP-korlát (percenként 5 próba) vagy lejárt munkamenet (419).

Tanúsítványhiba vagy lejárat

A `web` indításkor ellenőrzi a tanúsítványt. A `docker compose exec web openhdid tls-check` kiírja:

- az alanyt, a kiállítót és a lejáratot;
- a kulcs típusát és a láncot;
- a lefedett neveket és a hibát.

Csere: új fájlok a `tls` mappába (tulajdonos 82, a kulcs 400-as joggal), majd `docker compose exec web openhdid reload`. Ha a web nem fut, a javítás után `docker compose up -d`.

Nem mennek ki az e-mailek vagy az SMS-ek

1. *Tartalom › Kimenő üzenetek:* „várakozik” állapotnál a `worker` nem fut (`docker compose ps worker`); „sikertelen”-nél a sor hibaszövege mondja meg az okot.
2. Kapcsolatpróba az ügyfelek érintése nélkül: `docker compose exec web php artisan hdid:test-mail cim@example.org --now`, illetve `hdid:test-sms +36301234567 --now`.
3. Belső CA-val hitelesítő Exchange-nél a CA-fájl a `ca` mappába kerüljön, majd `docker compose restart web worker sync-worker scheduler` (a CA-kat a konténer induláskor olvassa be, a `docker compose up -d` változatlan beállításnál nem indít újra); a TLS-ellenőrzést ne kapcsolja ki.
4. A kimenő tűzfal engedje a konténerek felől az EWS/SMTP és az Ozeki címét.

Az ügyféltörzs-átvétel hibát jelez

Az *Ügyféltörzs-átvételek* lista hibaszövege dönt; az okok részletes listája a Rendszerállapot kártyájának tanácsában olvasható.

- **„Üres forrás” és „Gyanús forrás”:** a védőkorlát megállította a futást, senki nem zárult le. Ellenőrizze az export teljességét.
- **Hitelesítési hiba:** `docker compose exec web php artisan hdid:emd-sync-token`.
- **„Hibás belépési válasz”:** A belépés `POST` az `EMD_SYNC_LOGIN_URL` címre, JSON-törzsszel: `{"Password": "...", "Username": "..."};` a rendszer `2xx` választ vár, a törzs legfelső szintjén nem üres `access_token` és `refresh_token` mezővel. „Hibás belépési válasznál” a hibaszöveg a HTTP-státuszt és a kapott mezők nevét mondja (értékek nélkül), vagy azt, hogy a válasz nem JSON. Kézi próba ugyanazzal a kéréssel: megmutatja, hogyan érkezett meg a felhasználónév (JSON-ban egy backslash `\\` alakban látszik) és a jelszó (hossza, `=`-re végződik-e, van-e benne szóköz vagy idézőjel), majd a válasz státuszát és szerkezetét a tokenek értéke nélkül:

```

docker compose exec web php artisan tinker --execute '$u =
config("hdid.sync.username"); $p = (string) config("hdid.sync.password"); echo
"Username: ".json_encode($u).";
Password: ".strlen($p)." karakter; ==re végződik: ".(str_ends_with($p, "=") ? "igen" :
"nem")."; szóköz vagy idézőjel benne: ".(preg_match("/[\s\"']/", $p) ? "IGEN" :
"nem").";
Kérés: POST ".config("hdid.sync.login_url").";
".json_encode(["Password" => "***", "Username" => $u], JSON_UNESCAPED_UNICODE).";
"; $r = Illuminate\Support\Facades\Http::acceptJson()->asJson()->withoutRedirecting()-
>timeout(20)->post(config("hdid.sync.login_url"), ["Password" => $p, "Username" =>
$u]); echo "Válasz: HTTP ".$r->status().", ".$r->header("Content-Type").";
"; $walk = function ($v, $path) use (&$walk) { if (is_array($v)) { foreach ($v as $k =>
$x) { $walk($x, $path === "" ? (string) $k : "$path.$k"); } return; } echo " $path:
".gettype($v).(is_string($v) ? " (" .strlen($v)." karakter)" : "").";
"; }; $j = $r->json(); if (is_array($j)) { $walk($j, ""); } else { echo " nem JSON:
".mb_substr(trim(strip_tags($r->body())), 0, 200).";
"; }'

```

- **„Nem tudott kapcsolódni”** (belépés, ID-lista vagy export): a hibaszöveg zárójelben megnevezi a kiszolgálót és az okot. A kérés adatai nem kerülnek bele.
 - *a név a konténerből nem oldható fel (DNS)*: a konténer a gazdagép DNS-kiszolgálóit kérdezi, a gazdagép `/etc/hosts`-át nem látja. Csak ott szereplő névhez `extra_hosts` kell a `compose.override.yaml`-ban; helyi feloldás vagy VPN-es gazdagépnél a belső DNS-t a `/etc/docker/daemon.json` `dns` kulcsa adja (telepítési útmutató 6.7);
 - *a kapcsolatot elutasították, vagy a cím nem érhető el*: tűzfal a Docker-hálózat (a `docker network inspect openhdid_default` szerinti alhálózat) és a kiszolgáló között, útválasztás vagy rossz port;
 - *nem jött válasz időben*: jellemzően csendben eldobó tűzfal;
 - *a tanúsítvány nem ellenőrizhető* (OpenSSL 20, „unable to get local issuer certificate”): a kiszolgáló tanúsítványát belső tanúsítványkiadó írta alá. A lánc a gazdagépről:

```

openssl s_client -connect emd.ceg.local:443 -servername emd.ceg.local -showcerts
</dev/null 2>/dev/null | grep -E '^ *'[0-9]+ s:|^ *i:'

```

Az utolsó `i:` sor a hiányzó kiadó (jellemzően a vállalati gyökér-CA). A tanúsítványa PEM-formátumban (-----BEGIN CERTIFICATE-----; Windows-ból „Base-64 kódolású X.509”, DER-ből `openssl x509 -inform der -in ca.cer -out ca.crt`) a `ca` mappába kerül `.crt` vagy `.pem` kiterjesztéssel, mindenki számára olvashatóan (644); ha a kiszolgáló a köztes tanúsítványt sem küldi, az is. Ha a gazdagép maga már megbízik benne (a `curl https://emd.ceg.local/` a gazdagépről hiba nélkül fut), a gazdagép tanúsítványtárában (`/etc/pki/ca-trust/source/anchors/`) megvan. Utána `docker compose restart web worker sync-worker scheduler`; a `docker compose logs web | grep "extra CA"` mutatja, hány CA-fájlt vett fel.

A konténer szemszögéből minden beállított ügyféltörzs-címet egyszerre ellenőriz ez a parancs (DNS, TCP és HTTPS, a nyers hibaüzenettel):

```

docker compose exec web php artisan tinker --execute 'foreach
(["login_url","refresh_url","id_list_url","api_url"] as $k) { $u =
config("hdid.sync.$k"); if (! $u) { echo "$k: nincs beállítva

```

```

"; continue; } $h = parse_url($u, PHP_URL_HOST); $p = parse_url($u, PHP_URL_PORT) ? :
(parse_url($u, PHP_URL_SCHEME) === "http" ? 80 : 443); $ip = filter_var($h,
FILTER_VALIDATE_IP) ? $h : gethostbyname($h); echo "$k: $h:$p
DNS:   ".$ip === $h && ! filter_var($h, FILTER_VALIDATE_IP) ? "NEM OLDÓDIK FEL" :
$ip)."
"; $s = @fsockopen($ip, $p, $en, $es, 5); echo "   TCP:   ".$s ? "kapcsolódik" : "HIBA
($es)"."
"; try { $r = Illuminate\Support\Facades\Http::timeout(10)->connectTimeout(5)->get($u);
echo "   HTTPS: válaszol (HTTP ".$r->status().")
"; } catch (Throwable $e) { echo "   HTTPS: ".preg_replace("/ \\\(see .*\\$/s", "", $e-
->getMessage())."
"; } }'

```

- **Próba futtatás javítás után:** `docker compose exec web php artisan hdid:emd-sync --dry-run`.
- **Túl régi utolsó sikeres futás:** `docker compose exec scheduler php artisan schedule:list` mutatja, ütemezve van-e a `hdid:emd-sync ( EMD_SYNC_CRON )`.
- **E-mail-ütközés:** a Rendszerállapot sárga, a futás részleteinek „E-mail-ütközések” blokkja mutatja, kinek a címét nem lehetett átírni, mert másik fiókon áll; a fiók a régi címmel marad, és nem tud belépni, amíg az ütközést az ügyféltörzsben nem rendezik, vagy a másik fiókot nem törlik. A naplóban (`docker compose logs sync-worker`) figyelmeztető sor.
- **Besorolásváltás (ügyfél és munkatárs között):** a régi fiók csak annyi egymást követő futás után zárul le, mint egy hiányzó tétel; a függőben lévők a futás részletein látszanak. Megszűnő főügyfél-jogosultságnál a kapcsolatok magától megszűnnek, a kapcsolt ügyfelek nem zárulnak le.

A naplóban nem a kliensek IP-címe látszik

Ha az access log `ip` mezője minden kérésnél a Docker-átjáró vagy a terheléelosztó címe, az IP-alapú forgalomkorlát és az auditnapló címe hamis. A lehetséges okok és teendők:

- **IPv6-kapcsolat:** kapcsolja be a Docker IPv6-NAT-ját (`iptables`).
- **Terheléelosztó mögött:** `OPENHDID_TLS=off` és a `TRUSTED_PROXIES` a terheléelosztó címe (telepítési útmutató 6.5).
- **Userland-proxy:** kapcsolja ki a Docker-démonban (`"userland-proxy": false`).

Kiszolgáló kompromittálódása

1. **Elszigetelés:** izolálja a gazdagépet, és őrizze meg a naplókat.
2. **Új gép:** állítson fel tiszta gazdagépet, és a telepítést ellenőrzött kiadásból (aláírt image, 8. fejezet) és mentésből állítsa vissza.
3. **Titokcsere:** cserélje az összes titkot: adatbázis, EWS, Ozeki, SSO, API-kulcsok, ügyféltörzs, TLS-kulcs.
4. **Visszavonások:** vonja vissza a munkameneteket és a tokeneket. Az `APP_KEY` cseréje csak külön terv szerint történhet (5. fejezet).

7 Mentés és visszaállítás

A kiadás `backup.sh` szkriptje egy paranccsal ment és állít vissza. A telepítési mappában, a `compose.yaml` mellett él, bárholnan indítható, és root nélkül fut: sem `sudo`, sem root jogú konténer nem kell, rootful és rootless Dockerrel egyaránt működik. Egy mentés egyszerre fogja össze azt, ami a helyreállításhoz kell:

1. **az adatbázist** (a beépített MariaDB-ét, konzisztens pillanatképként);
2. **a storage kötetet** (arculati képek, feltöltések);
3. **a beállításokat**: `.env`, `db.env` (benne az `APP_KEY` és minden jelszó), `compose.yaml`, `compose.override.yaml`, a `ca` mappa;
4. **a tls mappát** (tanúsítvány és kulcs).

```
cd /opt/openhdid
./backup.sh                # mentés: backup/2026-10-09_023000
./backup.sh list           # a mentések listája (verzió, adatbázis, méret)
./backup.sh restore        # visszaállítás a legutóbbi mentésből
./backup.sh restore 2026-10-09_023000
```

A mentés. A szkript a saját mappájában létrehozza a `backup` almappát (700-as jog), és abba egy időbélyeges mappát: `database.sql.gz`, `storage.tar.gz`, `config.tar.gz`, `tls.tar.gz`, `info.txt` (időpont, OpenHDID-verzió, image) és `SHA256SUMS`; minden fájl 600-as jogú. A mentés futó vermen készül (a `mariadb` szolgáltatásnak futnia kell), kiesés nélkül. Alapból a legutóbbi 14 mentés marad, a régebbieket a szkript törli (`--keep N`, `0` = mind marad). A `tls` kulcsát az alkalmazás saját felhasználója olvassa ki, mert az üzemeltető számára nem olvasható.

Ütemezve (cron, naponta 2:30-kor; a `--quiet` csak hibánál ír, a cron ezt levélben küldi):

```
30 2 * * * /opt/openhdid/backup.sh --quiet
```

Visszaállítás

A `./backup.sh restore [név]` egy megerősítés (`igen`) után végigviszi a teljes helyreállítást; nem interaktív futtatáshoz `--yes`. A lépések:

1. **Ellenőrzés**: a mentés ellenőrzőösszegei; sérült vagy módosított mentésből nem állít vissza.
2. **Biztonsági mentés** a jelenlegi állapotról (`...visszaallitas-elott`), így a visszaállítás is visszavonható. Ha a jelenlegi verem már nem menthető, `--no-safety-backup`.
3. **Az alkalmazás leáll** (`web`, `worker`, `sync-worker`, `scheduler`); innentől a végéig kiesés van.
4. **Beállítások és tls**: a mentett `.env` és a többi fájl kerül vissza, a mentés `OPENHDID_VERSION`-jével együtt; a kulcsot az alkalmazás felhasználója írja vissza, a mappa erre a pillanatra nyílik meg.
5. **Adatbázis**: a táblák törlése, majd a mentés visszatöltése a migrációs fiókkal.
6. **Storage kötet**, majd **indítás** és rendszerállapot.

Ellenőrzés utána: belépés; egy ügyfél adatlapján a válaszok száma és egy kérdés-válasz próba. Ez bizonyítja, hogy a titkosított adatok az `APP_KEY`-jel olvashatók.

Új gépre. Docker telepítése (telepítési útmutató 2. fejezet), a telepítési mappa létrehozása, benne a kiadás `compose.yaml` és `backup.sh` fájlja és a `tls`, `ca` mappa (`mkdir -p tls ca`), majd a mentés mappájának átmásolása a `backup` alá, végül `./backup.sh restore <név>`. A beépített adatbázist és a két fiókot az első induló `mariadb` a visszaállított jelszavakkal hozza létre.

Követelmények a mentéssel szemben.

- A `backup` mappát rendszeresen másolja a gazdagépen kívülre, titkosítva és módosítás ellen védve; a helyi mentés a gép elvesztése ellen nem véd.
- A `config.tar.gz` (kulcsok, jelszavak) más hozzáférési szabállyal kerüljön tárolásra, mint az adatbázis.
- Határozza meg a megengedett adatvesztést (RPO) és a helyreállítási időt (RTO), és rendszeres visszaállítási próbával mérje; a próba egy másik gépen a fenti „Új gépre” lépésekkel végezhető.
- Külső adatbázisnál (a `db` profil nélkül) az adatbázist az adatbázis üzemeltetője menti és állítja vissza, a mentéssel egyidejű állapotra; a szkript a többit kezeli, és ezt az `info.txt`-ben jelzi.

8 Frissítés és visszaállítás

Kiadások. A kiadások a [GitHub Releases](#) oldalon jelennek meg, jegyzettel, dokumentumokkal és ellenőrzőösszegekkal.

- **Verziózás:** SemVer. A javítóverzió (1.0.x) hibajavítást vagy platformfrissítést hoz (PHP, nginx, OpenSSL, Alpine), az alverzió (1.x) új funkciót. Ha a főverzió változik, a jegyzet leírja a teendőket.
- **Címkék:** egy kiadott címke soha nem változik.

1. Az image ellenőrzése (választható, ajánlott):

```
gh attestation verify oci://ghcr.io/adamfhun/openhdid:1.4.2 --owner adamfhun
cosign verify ghcr.io/adamfhun/openhdid:1.4.2 \
  --certificate-identity-regexp
'^https://github.com/adamfhun/openhdid/.github/workflows/release\.yaml@refs/tags/v' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

2. Mentés: `./backup.sh` (7. fejezet).

3. Frissítés: az `OPENHDID_VERSION` átírása, majd `docker compose pull && docker compose up -d`. A migrációk automatikusan lefutnak, a szolgáltatások az új image-dzsel indulnak.

4. Ellenőrzés: `docker compose ps`, Rendszerállapot (a fejlécben az új verzió), a telepítési útmutató 12. fejezetének próbái.

Visszaállítás sikertelen frissítés után

1. Ha nem futott migráció: az `OPENHDID_VERSION` visszaírása az előző verzióra, majd `docker compose up -d`.

2. Ha futott migráció: a migrációk csak előre futnak. Állítsa vissza a frissítés előtti mentést:

`./backup.sh restore <név>` (7. fejezet); ez a mentett `.env`-vel az előző verziót is visszahozza és elindítja. A mentés óta keletkezett adatok elvesznek.

9 Skálázás és teljesítmény

A helyes méret mérésből jön, nem becslésből. Mielőtt bővít, mérje meg a jellemző terhelést a saját környezetében (például k6-tal), és rögzítse a céljait (például p95 < 300 ms). A tipikus forgatókönyvek:

- hívásesemények (`POST /api/v1/callcenter/calls`);
- IVR-kóddellenőrzés;
- portálbelépés;
- a vezérlőpult betöltése sok egyidejű ügyintézővel.

1. fokozat – egy kiszolgáló, alapbeállítás

Egy `web` 8 PHP-FPM folyamattal, egy-egy feldolgozó, beépített MariaDB, adatbázis-alapú munkamenet, gyorsítótár és sor. Néhány ezer ügyfél és napi több ezer hívás mellett elegendő.

2. fokozat – egy kiszolgáló, nagyobb terhelés

- **PHP-FPM:** `OPENHDID_FPM_MAX_CHILDREN` $\approx$ (a webkonténernek szánt memória – 150 MB) / 70 MB; `OPENHDID_FPM_PM=static` egyenletes terhelésnél.
- **MariaDB:** `OPENHDID_DB_BUFFER_POOL` a gép memóriájának 25–50%-a. A `OPENHDID_DB_MAX_CONNECTIONS` legyen legalább az összes FPM-folyamat + a feldolgozók + 20.
- **Valkey:** `COMPOSE_PROFILES=db,redis`, `SESSION_DRIVER=redis`, `CACHE_STORE=redis`, `QUEUE_CONNECTION=redis`, `REDIS_PASSWORD`. Ez leveszi a munkameneteket, a zárok és a sor terhet az adatbázisról. A Valkey a sor tartósságához AOF-naplót használ.
- **Feldolgozók:** `docker compose up -d --scale worker=3`. Az üzenetek egyszer mennek ki, több feldolgozó mellett is.

3. fokozat – több kiszolgáló

- **Terheléelosztó:** a TLS a terheléelosztón zárul le; a webpéldányokon `OPENHDID_TLS=off`, a `TRUSTED_PROXIES` a terheléelosztó címe.
- **Közös storage:** NFS vagy más, több gépről írható kötet. Az arculati képek és a panel fájlfeltöltései (a Livewire ideiglenes feltöltései) több webpéldány között csak így érhetők el.
- **Közös állapot:** Valkey kötelező; `APP_MAINTENANCE_DRIVER=cache`.
- **Ütemező:** egyetlen példány az egész telepítésben.
- **Adatbázis:** menedzselte vagy replikált MariaDB, egyetlen írható elsődleges példánnyal. Galera-fürtnél is egy írási csomópontot használjon; a kimutatások olvasási replikára terelhetők.

Kubernetes-megfeleltetés

SZEREP	KUBERNETES-ERŐFORRÁS
<code>web</code>	Deployment (több replika, HPA CPU alapján), Service, Ingress vagy terheléelosztó; <code>OPENHDID_TLS=off</code> . Élő- és készenléti próba: <code>exec openhdid healthcheck</code> .
<code>worker</code> , <code>sync-worker</code>	Deployment; a replikaszáma a sorhossz szerint is skálázható.

SZEREP	KUBERNETES-ERŐFORRÁS
<code>scheduler</code>	Deployment 1 replikával, <code>Recreate</code> stratégiával.
<code>migrate</code>	Job, a web frissítése előtt (például Helm pre-upgrade hook).
<code>storage</code>	ReadWriteMany PVC (NFS, CephFS).
titkok	Secret fájlként csatolva + <code>*_FILE</code> változók.
biztonsági környezet	<code>runAsNonRoot</code> , <code>runAsUser: 82</code> , <code>readOnlyRootFilesystem</code> , <code>allowPrivilegeEscalation: false</code> , <code>capabilities.drop: [ALL]</code> , <code>seccompProfile: RuntimeDefault</code> ; <code>/tmp</code> memóriában lévő <code>emptyDir</code> -ként.
MariaDB, Valkey	menedzselt szolgáltatás vagy operátor.

Az Octane-szerű, memóriában maradó PHP-futtatás tovább növelhetné az áteresztőképességet, de az nginx + PHP-FPM modellt váltaná ki, és kódatvizsgálatot igényel. Csak méréssel igazolt igény esetén jöhet szóba.

10 Biztonsági karbantartás

GYAKORISÁG	TEENDŐ
Folyamatosan	<code>/health</code> , konténerállapot, tanúsítványlejárat, kiugró 4xx/5xx, sikertelen belépések és kulcshasználat; a riasztás próbával igazolt felelőshöz érkezzen.
Hetente	A kiadott image-et a projekt hetente automatikusan átvizsgálja (Trivy). Ha javítható, magas kockázatú lelet van, javítóverzió készül, és a kiadási jegyzet jelzi. Kövesse a kiadásokat (GitHub „Watch › Releases”), és a gazdagép OS- és Docker-frissítéseit is.
Minden frissítéskor	Mentés, aláírás-ellenőrzés, a telepítési útmutató 12. fejezetének próbái.
Negyedévente	Hozzáférések, titkok és gépi kulcsok felülvizsgálata, mentés-visszaállítási próba, tűzfalszabályok, az OWASP-eltérések (telepítési útmutató 12.3) állapota.

Támogatási naptár (a 2026. szeptemberi állapot):

- PHP 8.5 aktív támogatás 2027. 12. 31-ig;
- Alpine 3.24 2028. 05.-ig;
- OpenSSL 3.5 LTS 2030-ig;
- MariaDB 12.3 LTS 2029. 06.-ig;
- a buildhez használt Node.js 2026. 10. 28-tól a 26 LTS ágra vált.

A projekt a lejárat előtt javítóverzióban lép a következő ágra, és a kiadási jegyzet jelzi.

Az image védelmei:

- nem root felhasználó, csak olvasható fájlrendszer, elvett Linux-képességek, `no-new-privileges` ;
- digesttel rögzített alapimage-ek, aláírt image, SBOM.

A `compose.yaml` biztonsági beállításait (`read_only` , `cap_drop` , `security_opt`) ne lazítsa.

11 Adatmegőrzés

A megőrzést két felületi beállítás szabályozza (*Beállítások* › *Adatmegőrzés*):

- **általános megőrzés** (alapból 5 év): auditnapló, azonosítási munkamenetek, hívások, kimenő üzenetek, átvételek;
- **rövid életű tételek** (alapból 2 év): lejárt egyszeri kódok, feltöltött állományok.

A napi 03:30-as tisztítás véglegesen töröl; az eredmény `retention.pruned` eseményként az auditnaplóban áll. A titkot hordozó üzenetek törzse már a kiküldés, a végleges hiba vagy a visszavonás után törlődik. A mentések megőrzését a mentési rend külön szabályozza: az alkalmazás nem törli őket.

12 Parancsreferencia

Az alkalmazásparancsok (`hdid:*`) a `docker compose exec web php artisan` előtaggal futnak.

PARANCSS	MIT TESZ
<code>./backup.sh / list / restore [név]</code>	Mentés a <code>backup</code> mappába / a mentések listája / visszaállítás (7. fejezet).
<code>docker compose run --rm --no-deps web key</code>	Új <code>APP_KEY</code> kiírása, csak új telepítéshez.
<code>docker compose exec web openhdid tls-check / reload / version</code>	Tanúsítvány ellenőrzése / nginx újratöltése ellenőrzéssel / verzió.
<code>hdid:make-admin email [--name=]</code>	Főrendszergazda létrehozása vagy előléptetése (jelszót kér).
<code>hdid:health [--strict] [--json]</code>	Rendszerállapot; kilépési kód 0/1/2.
<code>hdid:emd-sync [--file=] [--dry-run]</code>	Ügyféltörzs-átvétel (a régi <code>hdid:sync</code> név álnévként működik).
<code>hdid:emd-sync-token</code>	Az ügyféltörzs API-tokenjének frissítése, szükség esetén új belépés.
<code>hdid:test-mail cím [--now]</code> , <code>hdid:test-sms szám [--now]</code>	Próbaüzenet; <code>--now</code> a sor megkerülésével.
<code>hdid:api-key:create "Név" callcenter mobile_backend</code>	Gépi kulcs; egyszer jelenik meg.
<code>hdid:messages:flush</code>	Várakozó üzenetek azonnali küldése.
<code>hdid:prune</code>	Adatmegőrzési tisztítás kézzel.
<code>schedule:list</code> (a <code>scheduler</code> konténerben)	Ütemezett feladatok és következő futásuk.

PARANCS

MIT TESZ

```
queue:failed , queue:retry all
```

Sikertelen feladatok listája, újrapróbálása a hiba javítása után.

```
down --retry=30 , up
```

Karbantartási mód be és ki (minden példányra).

Rövidítésjegyzék

RÖVIDÍTÉS

JELENTÉS

EMD

Enterprise Master Data; magyarul ügyféltörzs.

AOF

Append Only File – a Valkey tartós naplója.

FPM

FastCGI Process Manager – a PHP folyamatkezelője.

HPA

Horizontal Pod Autoscaler – a Kubernetes automatikus vízszintes skálázója.

LTS

Long-Term Support – hosszú támogatású kiadási ág.

PVC

PersistentVolumeClaim – tartós kötet igénylése Kubernetesben.

RPO / RTO

Recovery Point Objective / Recovery Time Objective – megengedett adatvesztés és helyreállítási idő.

SBOM

Software Bill of Materials – az image összetevőinek jegyzéke.

SemVer

Semantic Versioning – szemantikus verziózás (fő.al.javító).